

Metodologie di Programmazione (A-L)

Il semestre - a.a. 2025 – 2026

Classi Annidate

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Classi annidate e interne

<http://docs.oracle.com/javase/tutorial/java/javaOO/nested.html>

Classi top-level

- Le classi usate finora vengono dette **top-level**, cioè esse si trovano più **in alto** di tutte le altre e non sono contenute in altre classi
- Questo tipo di classi richiede un file `.java` **dedicato** con lo stesso nome della classe che esso contiene



Classi annidate (nested class)

- Java consente di scrivere **classi** all'interno di altre **classi**
- Le classi presenti all'interno sono chiamate **classi annidate** (nested class)
- Possono essere **static**
- Oppure **non-static**: in questo caso vengono dette **classi interne** (inner class)



Classi interne (inner class) – quelle non statiche

- Prima di poter creare un oggetto della classe interna è necessario istanziare la classe esterna (top-level) che la contiene
- Ciascuna classe interna, infatti, ha un **referimento implicito** all'oggetto della classe che la contiene
- Dalla classe interna è possibile accedere a **tutte le variabili** e a **tutti i metodi** della classe esterna
- Come tutti i membri di una classe, le classi interne possono essere dichiarate **public**, **protected** o **private**

Classi interne (inner class) 1

- L'accesso a campi e metodi sia dell'oggetto della classe interna che dell'oggetto della classe esterna a cui è legato a doppio filo avviene normalmente
- Per disambiguare casi di ambiguità (es. campi con lo stesso nome nella classe interna ed esterna):
 - Se dalla classe interna viene usato soltanto **this** si fa riferimento ai campi e ai metodi di quella classe
 - Per far riferimento alla classe esterna è necessario far precedere **this** dal nome della classe esterna e il punto

Classi interne (inner class) 2

- Per istanziare la classe interna dalla classe esterna è sufficiente utilizzare l'operatore `new`
- Per istanziare la classe interna a partire da un'altra classe si utilizza la sintassi `referimentoOggettoClasseEsterna.new ClasseInterna()`

Classi interne (inner class)

```
public class Tastiera
{
    private String tipo;
    private Tasto[] tasti;

    public class Tasto
    {
        private char c;

        public Tasto(char c)
        {
            this.c = c;
        }

        public char premi()
        {
            return c;
        }

        public String toString()
        {
            return Tastiera.this.tipo + ": " + premi();
        }
    }

    public Tastiera(String tipo, char[] caratteri)
    {
        this.tipo = tipo;
        tasti = new Tasto[caratteri.length];

        for(int i=0; i<caratteri.length; i++)
            tasti[i] = new Tasto(caratteri[i]);
    }
}
```

La classe Tasto è una **inner-class** (classe interna) quindi non statica

Tasto ha **accesso** al campo (**privato!**) della classe esterna

Posso anche accedere al campo direttamente tramite il nome (tipo, invece di Tastiera.this.tipo)

Classi annidate statiche (static nested class)

- Se invece la classe all'interno è statica allora essa **non richiede** l'esistenza di un oggetto istanza della classe esterna e **non ha nemmeno un riferimento implicito** ad essa
 - Come con i metodi statici, non può accedere allo stato degli oggetti istanza della classe esterna.
- Da un punto di vista di **comportamento**, una classe annidata statica è equivalente ad una classe top-level inserita all'interno di un'altra classe top-level.
- Sono accessibili tramite il nome della classe esterna che le contiene, secondo la forma `new ClasseEsterna.ClasseAnnidataStatica()`

Classi annidate statiche (static nested class)

```
public class Tastiera
{
    private String tipo;
    private Tasto[] tasti;

    public static class Tasto
    {
        private char c;

        public Tasto(char c)
        {
            this.c = c;
        }

        public char premi()
        {
            return c;
        }

        public String toString()
        {
            return Tastiera.this.tipo + ": " + premi();
        }
    }

    public Tastiera(String tipo, char[] caratteri)
    {
        this.tipo = tipo;
        tasti = new Tasto[caratteri.length];

        for(int i=0; i<caratteri.length; i++)
            tasti[i] = new Tasto(caratteri[i]);
    }
}
```

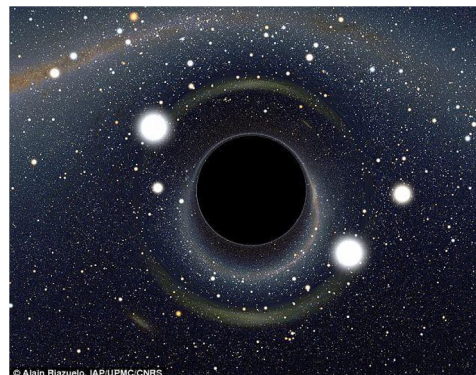
Errore in fase di **compilazione**: la classe static Tasto non ha **accesso** implicito ai membri della classe esterna

Ma la classe è visibile all'esterno

```
public class Computer
{
    public static void main(String[] args)
    {
        Tastiera.Tasto tasto = new Tastiera.Tasto('f');
        tasto.premi();
    }
}
```

Classi annidate: perché sono utili?

- Raggruppamento **logico** delle classi
 - Se una classe è utile solo ad un'altra classe, è logico inserirla al suo interno e tenere le due classi **logicamente vicine**
- Incrementa **l'incapsulamento**
 - Una classe B annidata in A può accedere ai membri di A (anche se **privati**), ma B può essere nascosta al mondo esterno
- Codice più **leggibile** e più facile da **mantenere**
 - La **vicinanza spaziale** è un fattore decisivo



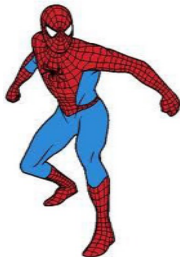


Esercizio: Disney vs. Marvel (1)

- Modellare uno scontro su **campo** tra **personaggi Disney** e **personaggi Marvel**
- **Squadra Disney = Paperinik, Ciccionik, Superpippo**
 - Paperinik, Ciccionik e Superpippo sono la **versione supereroe** dei corrispettivi personaggi (Paperino, Ciccio e Pippo)



- **Squadra Marvel = Spiderman, La cosa, Magneto**
 - Notare che Spiderman è la **versione supereroe** di Peter Parker, noto fotografo del Daily Bugle; La cosa e Magneto invece sono supereroi 24h





Esercizio: Disney vs. Marvel (2)

- I personaggi con una doppia vita devono esporre un'interfaccia DoppiaVita con i seguenti metodi:
 - `assumildentitaSegreta()`: consente di trasformarsi in supereroe
 - `assumildentitaPubblica()`: consente di assumere le sembianze 'umane'
- Ogni supereroe implementa l'interfaccia Supereroe con:
 - `attacca()`: sferra l'attacco specifico del supereroe
- Creare una partita su `campo` (ovvero una classe di test) in cui le due squadre si alternano in attacchi micidiali
 - Il nemico viene scelto casualmente dalla lista dei supereroi avversari



Esercizio: Disney vs. Marvel (3)

- Hint per **versione semplificata**: Senza utilizzare classi interne, ma solo interfacce ed ereditarietà
- Hint per **versione elaborata**: Per riflettere il fatto che nessuno è a conoscenza dei superpoteri in possesso dalle controparti umane, i supereroi, laddove possibile, dovrebbero estendere le classi umane corrispondenti ed essere implementati come loro **classi interne private** (ad es. Paperinik estende Paperino, ma solo Paperino potrà restituire un'istanza di Paperinik tramite l'interfaccia DoppiaVita)



Classi anonime
Interfacce Funzionali

Classi anonime

- E' possibile definire **classi anonime** (ovvero **senza nome**) che implementano un'interfaccia o estendono una classe
- Utilizzate esclusivamente per creare **un'unica istanza**
- Utili in determinati contesti (ad esempio, per creare un **oggetto al volo senza dover definire una classe**)
- Sintassi:

```
TipoDaEstendere unicoRiferimentoAOggetto = new TipoDaEstendere()  
{  
    // codice della classe anonima (implementazione dell'interfaccia  
    // o estensione della classe)  
};
```

Esempio: l'interfaccia Formula

Consideriamo la seguente interfaccia:

```
public interface Formula {  
    double calculate(int a);  
    default double sqrt(int a) { return Math.sqrt(a); }  
}
```



Esempio:
una classe che implementa l'interfaccia

```
public class ImplFormula implements Formula{  
  
    @Override  
    public double calculate(int a) {  
        return a+2.0;  
    }  
  
}
```



Esempio:
una classe anonima che implementa
l'interfaccia

```
Formula formula = new Formula() {  
    @Override  
    public double calculate(int a) {  
        return a+2.0;  
    }  
};  
formula.calculate(10);  
formula.sqrt(100);
```

Interfacce funzionali

- Da Java 8 è disponibile una nuova annotazione `@FunctionalInterface`
- L'annotazione `garantisce` che l'interfaccia non possa avere più di un metodo astratto
- Un' interfaccia funzionale ha un solo metodo astratto, ma può avere molteplici metodi `default`.
- Ad esempio:

```
@FunctionalInterface
public interface Runnable
{
    void run();
}
```


Single Abstract Method (SAM) type

- Le interfacce funzionali sono dette anche **SAM**
- **Single**
- **Abstract**
- **Method**

Interfacce funzionali

Supponiamo di voler fare una funzione che applichi un'operazione tra due numeri

```
public static int calcola(int a, int b, String tipo) {  
    if (tipo.equals("somma")) {  
        return a + b;  
    } else if (tipo.equals("prodotto")) {  
        return a * b;  
    } else if (tipo.equals("max")) {  
        return Math.max(a, b);  
    }  
}
```

Interfacce funzionali

Supponiamo di voler fare una funzione che applichi un'operazione tra due numeri

```
public static int calcola(int a, int b, String tipo) {  
    if (tipo.equals("somma")) {  
        return a + b;  
    } else if (tipo.equals("prodotto")) {  
        return a * b;  
    } else if (tipo.equals("max")) {  
        return Math.max(a, b);  
    }  
}
```

- Necessario modificare il metodo ogni volta che si vuole aggiungere un comportamento
- Codice poco pulito (if/else ovunque)
- Nessun controllo a compile-time (scrivi "sommaa" → errore a runtime)
- Non è estensibile

Passare funzioni in input mediante le interfacce

- Le interfacce permettono il passaggio in input di funzioni con una determinata intestazione.
- In Java non possiamo passare funzioni direttamente, ma oggetti che le rappresentano.
- Le interfacce possono essere utilizzate per definire la forma di queste funzioni, così possiamo passarle come parametri.

Passare funzioni in input mediante le interfacce

Possiamo definire una interfaccia

```
interface Operazione {  
    int esegui(int a, int b);  
}
```

E creare una classe che la implementa

```
class Somma implements Operazione {  
    public int esegui(int a, int b) {  
        return a + b;  
    }  
}
```

Riscrivere il metodo come

```
public static int calcola(int a, int b, Operazione op) {  
    return op.esegui(a, b);  
}
```

Passare funzioni in input mediante le interfacce

Chiamare la funzione

```
public static void main(String[] args) {  
    int r1 = calcola(5, 3, new Somma());  
    System.out.println(r1);  
}
```

Espressioni lambda



- In Java 8 è possibile specificare funzioni utilizzando una notazione molto compatta, le **espressioni lambda**:

`() -> { System.out.println("hello, lambda!"); }`

- Tali espressioni creano oggetti anonimi assegnabili a riferimenti ad **interfacce funzionali compatibili con l'intestazione** (input/output) della funzione creata

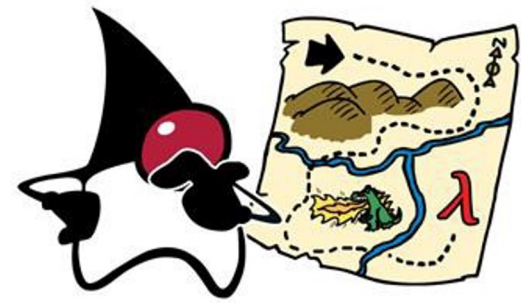
Sintassi delle espressioni lambda

- Un'espressione lambda è definita come segue:

`(tipo1 nome_param1, ..., tipon nome_paramn) -> { codice della funzione }`

- Il **tipo dei parametri** in input è opzionale, perché desunto dal contesto (a quale interfaccia funzionale facciamo riferimento?)
- Le **parentesi tonde** sono opzionali se in input abbiamo un solo parametro
- Le **parentesi graffe** intorno al codice della funzione sono opzionali se è costituito da una sola riga
 - Non è necessario **return**, se il codice è dato dall'espressione di ritorno

Espressioni lambda



```
@FunctionalInterface  
public interface Runnable  
{  
    void run();  
}
```

```
Runnable r = () -> { System.out.println("hello, lambda!"); };  
r.run(); // stampa "hello, lambda!"
```



Espressioni lambda equivalenti

- `(int a, int b) -> { return a+b; }`
- `(a, b) -> { return a+b; }`
- `(a, b) -> return a+b;`
- `(a, b) -> a+b;`

- `(String s) -> { return s.replace(' ', '_'); }`
- `(s) -> { return s.replace(' ', '_'); }`
- `s -> { return s.replace(' ', '_'); }`
- `s -> return s.replace(' ', '_');`
- `s -> s.replace(' ', '_');`





Esempi di espressioni lambda

- $(\text{int } a, \text{int } b) \rightarrow a+b$
- $a \rightarrow a*a$
- $() \rightarrow \text{System.out.println("Hello, world!");}$
- $s \rightarrow \text{System.out.println}(s);$
- $() \rightarrow 42;$
- $() \rightarrow 3.1428;$
- $(\text{String } s) \rightarrow s.\text{length}();$
- $s \rightarrow s.\text{length}();$



Consideriamo nuovamente il nostro esempio

```
public interface Formula
{
    double calculate(int a);
}
```

Esempio di implementazione di Formula prima e dopo Java 8

In Java 7:

```
Formula formula = new Formula() {  
    @Override  
    public double calculate(int a)  
    {  
        return Math.sqrt(a * 100);  
    }  
};
```

```
public interface Formula{  
    double calculate(int a);  
    default double sqrt(int a) {  
        return Math.sqrt(a);  
    }  
}
```

In Java 8:

```
Formula formula = a -> Math.sqrt(a*100);  
Formula formula2 = a -> a*a;  
Formula formula3 = a -> a-1;
```



Esempio: conversione da un tipo F a un tipo T

@FunctionalInterface

```
public interface Converter<F, T>
```

```
{
```

```
    T convert(F from);
```

```
}
```

```
Converter<String, Integer> converter = from -> Integer.valueOf(from);
```

```
Integer converted = converter.convert("123");
```

```
Converter<String, MyString> stringConverter = a -> new MyString(a);
```

```
MyString myString = stringConverter.convert("123");
```

Differenze tra classi anonime ed espressioni lambda

- La parola chiave **this**:
 - **Classi anonime**: si riferisce all'oggetto anonimo

```
public class Test {  
    void stampa() {  
        Runnable r = new Runnable() {  
            public void run() {  
                System.out.println(this.getClass().getName());  
            }  
        };  
        r.run();  
    }  
}
```

Output: Test\$1

- **Espressioni lambda**: si riferisce all'oggetto della classe che le racchiude

```
public class Test {  
    void stampa() {  
        Runnable r = () -> {  
            System.out.println(this.getClass().getName());  
        };  
        r.run();  
    }  
}
```

Output: Test

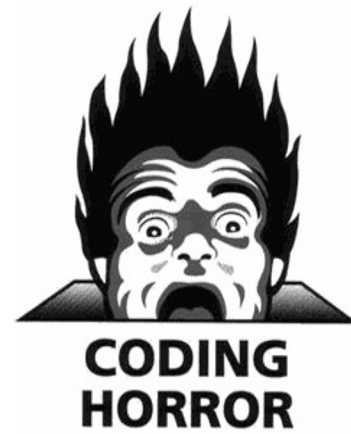
Differenze tra classi anonime ed espressioni lambda

- La compilazione è differente:
 - **Classi anonime:** compilate come classi interne
 - **Espressioni lambda:** compilate come metodi privati invocati dinamicamente

Quando utilizzare le lambda?

- E' da consigliare l'impiego delle espressioni lambda **principalmente** quando il codice si scrive su una sola riga
- In alternativa, si preferisce un'implementazione mediante classe o classe anonima.

Coding horror



- Due problemi qui:
 - If (bAcceso) ecc. si trasforma con l'operatore Elvis in una sola riga di codice
 - non usa costanti per "ACCESO" e "SPENTO"

```
public class ErroriComuni
{
    private boolean bAcceso;

    @Override
    public String toString()
    {
        if (bAcceso) return "ACCESO";
        else return "SPENTO";
    }
}
```

Coding horror

- Soluzione corretta:



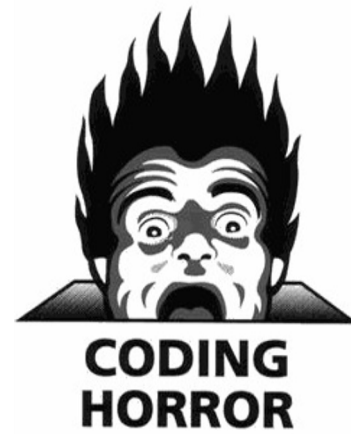
**CODING
HORROR**

```
public class ErroriComuni
{
    public static final String ACCESO = "accesso";
    public static final String SPENTO = "spento";

    private boolean bAcceso;

    @Override
    public String toString()
    {
        return bAcceso ? ACCESO : SPENTO;
    }
}
```

Coding horror



- Altre forme assolutamente ineleganti:

```
if (bAcceso == true) return true;  
else return false;
```

```
return bAcceso ? true : false;
```

- Ancora:

```
if (bAcceso == true) System.out.println("vero");  
if (bAcceso == false) System.out.println("falso");
```

- Si usa invece `if (bAcceso)` e, con l'operatore di negazione `!`, `if (!bAcceso)`

Coding horror

- Utilizzo del break o del return per uscire da un ciclo
 - Va evitato il booleano "d'appoggio" se non serve dopo!

```
public boolean cercaNO(int k, int... elenco)
{
    boolean trovato = false;

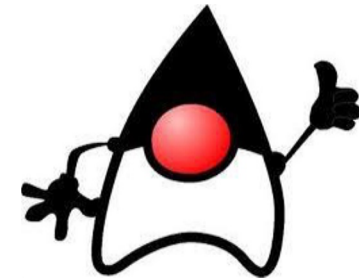
    for (int x : elenco)
        if (x == k) trovato = true;

    return trovato;
}
```



```
public boolean cercaOK(int k, int... elenco)
{
    for (int x : elenco)
        if (x == k) return true;

    return false;
}
```



Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.